МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЗАХІДНОУКРАЇНСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
Факультет комп’ютерних інформаційних технологій
Кафедра комп’ютерних наук

ЗВІТ

про виконання тренінгу та самостійної роботи
з дисципліни «Управління та стратегічний розвиток ІТ-бізнесу»
на тему: «Розробка ПЗ для управління командою. Варіант 1: Task Manager для команди»

Виконав:
студент групи ІПЗМПм-11
Боднар А.

Перевірив:

Тернопіль – 2026

МЕТА РОБОТИ

Навчитися моделювати об’єкти та процеси командної роботи, визначати показники пріоритетності задач, оцінювати навантаження членів команди та формувати аналітичні звіти для менеджера; розробити програмний засіб для управління задачами команди.

ЗАВДАННЯ (ВАРІАНТ 1)

Розробити Task Manager для команди з такими можливостями:

– створення, редагування та видалення задач;
– призначення задач на учасників команди;
– встановлення дедлайнів, важливості та складності задач;
– автоматичне обчислення пріоритету задач і відображення списку за пріоритетом;
– фільтрування задач за статусом (в роботі, завершено, прострочено) і пріоритетом;
– формування короткого звіту по задачах команди (топ-3 задачі, прострочені задачі).

ХІД РОБОТИ

1. Модель даних

Задача описується назвою, відповідальним, дедлайном, важливістю (1–5), складністю (1–5) та ознакою завершення. Статус обчислюється автоматично: «Завершено» — якщо задачу позначено виконаною, «Прострочено» — якщо дедлайн минув, інакше — «В роботі».

2. Формула пріоритету

Пріоритет задачі — зважена сума трьох нормованих факторів:

$$P = 100 \cdot (0{,}45 \cdot I + 0{,}2 \cdot S + 0{,}35 \cdot U),$$

де I = (важливість − 1)/4 і S = (складність − 1)/4 — важливість і складність, приведені до шкали 0…1; U — терміновість: 1, якщо до дедлайну 0 днів або дедлайн минув, 0 — якщо до дедлайну 14 днів і більше, а між цими значеннями змінюється лінійно. Найбільшу вагу має важливість, далі — терміновість, найменшу — складність. Пріоритет від 65 вважається високим, від 40 — середнім, нижче — низьким; завершені задачі мають пріоритет 0 і відображаються в кінці списку.

3. Звіт для менеджера

Звіт містить кількість задач за статусами, частку виконаних задач, три задачі з найвищим пріоритетом, перелік прострочених задач і навантаження кожного учасника (кількість незавершених і прострочених задач, сума балів складності).

4. Програмна реалізація

Програму реалізовано як вебзастосунок мовою TypeScript з бібліотеками React і React-Bootstrap; вона є розділом сайту з навчальними роботами. Логіку винесено в модуль tasks.ts (статус, терміновість, пріоритет, сортування, звіт, перевірка введення), інтерфейс — у TaskManager.tsx. Задачі зберігаються в локальному сховищі браузера, тож дані не губляться після перезавантаження сторінки. Правильність формул перевірено автоматичними тестами.

Сторінка роботи на сайті: https://4ndruxa.github.io/university-labs/#/usr/tm.

Репозиторій з кодом: https://github.com/4ndruxa/university-labs.

5. Результати роботи програми

На рисунку 1 наведено список задач команди з пріоритетами, статусами та кнопками дій.

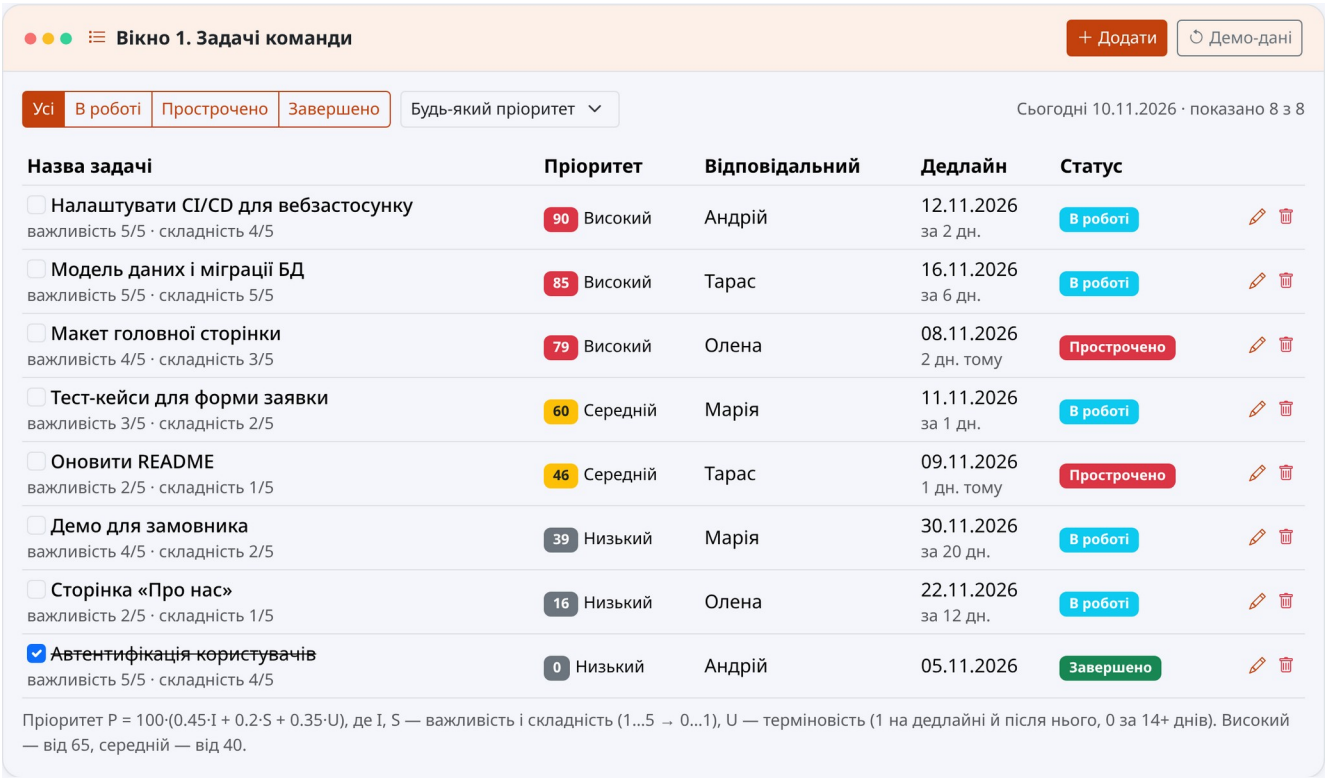


Рисунок 1 – Список задач команди

На рисунку 2 наведено звіт по задачах команди.

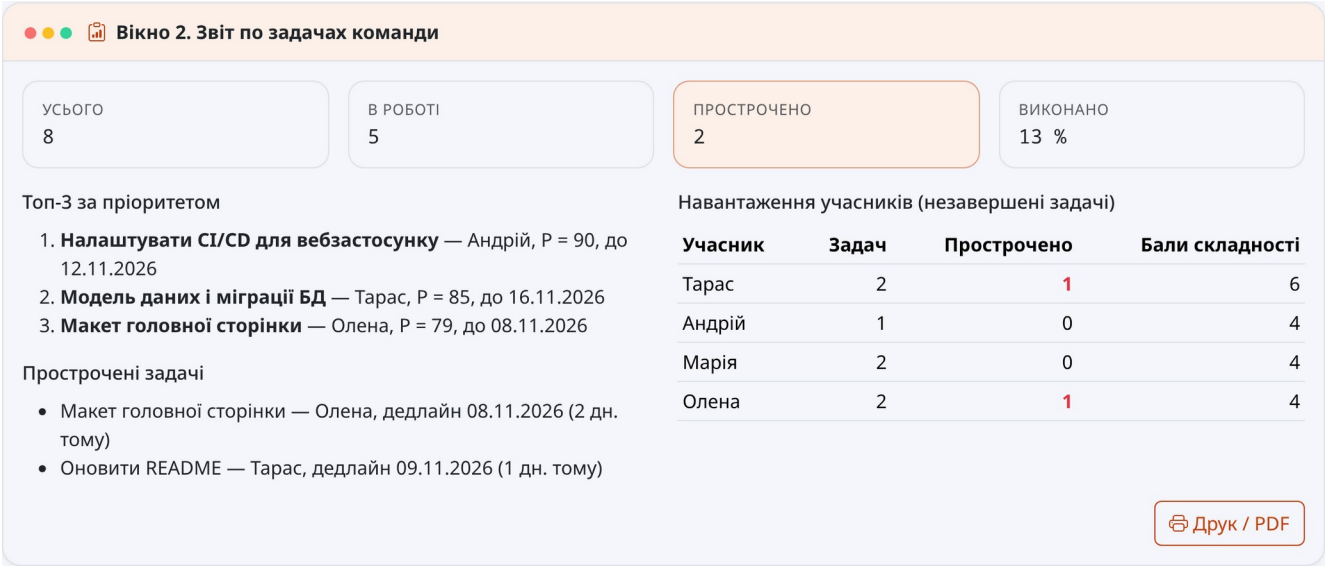


Рисунок 2 – Звіт по задачах команди

ВИСНОВКИ

Розроблено Task Manager для команди, який автоматично обчислює пріоритет задач з урахуванням важливості, складності й терміновості, відображає задачі за пріоритетом, фільтрує їх за статусом і пріоритетом та формує звіт для

менеджера. Така модель дозволяє швидко побачити найважливіші та прострочені задачі й нерівномірність навантаження в команді.

ДОДАТОК А. ЛІСТИНГ ОБЧИСЛЮВАЛЬНОЇ ЧАСТИНИ

Файл tasks.ts

```
// Тренінг (варіант 1): Task Manager для команди — модель задач, пріоритет,
    статуси, звіт
//
// Пріоритет P ∈ [0; 100] — зважена сума трьох нормованих факторів:
//   P = 100 · (0,45·I + 0,20·S + 0,35·U)
//   I = (важливість − 1) / 4,  S = (складність − 1) / 4  — шкала 1…5 → 0…1
//   U — терміновість: 1, якщо до дедлайну ≤ 0 днів; 0, якщо ≥ 14 днів; між ними —
    лінійно
// Завершені задачі мають пріоритет 0 і в списку йдуть останніми.


export type Status = 'inProgress' | 'done' | 'overdue'
export type Level = 'high' | 'medium' | 'low'


export interface Task {
  id: string
  title: string
  assignee: string
  deadline: string // YYYY-MM-DD
  importance: number // 1…5
  complexity: number // 1…5
  done: boolean
}


export const WEIGHTS = { importance: 0.45, complexity: 0.2, urgency: 0.35 } as
    const
export const URGENCY_HORIZON = 14 // днів


export const STATUS_LABEL: Record<Status, string> = { inProgress: 'В роботі',
    done: 'Завершено', overdue: 'Прострочено' }
export const LEVEL_LABEL: Record<Level, string> = { high: 'Високий', medium:
    'Середній', low: 'Низький' }


const DAY = 86_400_000
/** Дата YYYY-MM-DD → номер дня (UTC), щоб не залежати від часового поясу */
const dayNumber = (iso: string): number => Math.floor(Date.parse(`${iso}
    T00:00:00Z`) / DAY)
export const today = (now = new Date()): string =>
  `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, '0')}-$
    {String(now.getDate()).padStart(2, '0')}`


export const daysLeft = (task: Task, on: string): number =>
    dayNumber(task.deadline) - dayNumber(on)


export function status(task: Task, on: string): Status {
  if (task.done) return 'done'
  return daysLeft(task, on) < 0 ? 'overdue' : 'inProgress'
}


export function urgency(task: Task, on: string): number {
  const d = daysLeft(task, on)
  if (d <= 0) return 1
  if (d >= URGENCY_HORIZON) return 0
  return 1 - d / URGENCY_HORIZON
}
```

```
export function priority(task: Task, on: string): number {
  if (task.done) return 0
  const i = (task.importance - 1) / 4
  const s = (task.complexity - 1) / 4
  return Math.round(100 * (WEIGHTS.importance * i + WEIGHTS.complexity * s +
    WEIGHTS.urgency * urgency(task, on)))
}

export const level = (p: number): Level => (p >= 65 ? 'high' : p >= 40 ?
    'medium' : 'low')

export interface Row extends Task {
  priority: number
  level: Level
  status: Status
  daysLeft: number
}

/** Задачі з обчисленими полями, упорядковані за спаданням пріоритету (завершені —
    в кінці) */
export function rank(tasks: readonly Task[], on: string): Row[] {
  return tasks
    .map((t) => {
      const p = priority(t, on)
      return { ...t, priority: p, level: level(p), status: status(t, on),
    daysLeft: daysLeft(t, on) }
    })
    .sort((a, b) => Number(a.done) - Number(b.done) || b.priority - a.priority ||
    a.deadline.localeCompare(b.deadline))
}

export function validateTask(t: Omit<Task, 'id'>): string[] {
  const errors: string[] = []
  if (!t.title.trim()) errors.push('Вкажіть назву задачі')
  if (!t.assignee.trim()) errors.push('Призначте відповідального')
  if (!/^\d{4}-\d{2}-\d{2}$/.test(t.deadline) ||
    Number.isNaN(Date.parse(t.deadline))) errors.push('Вкажіть дедлайн')
  for (const [k, name] of [['importance', 'Важливість'], ['complexity',
    'Складність']] as const) {
    if (!Number.isInteger(t[k]) || t[k] < 1 || t[k] > 5) errors.push(`${name} —
    ціле число від 1 до 5`)
  }
  return errors
}

export interface Report {
  total: number
  byStatus: Record<Status, number>
  top3: Row[]
  overdue: Row[]
  /** Незавершені задачі на кожного учасника */
  load: { assignee: string; open: number; overdue: number; points: number }[]
  completion: number // частка завершених, %
}

export function report(tasks: readonly Task[], on: string): Report {
  const rows = rank(tasks, on)
  const byStatus: Record<Status, number> = { inProgress: 0, done: 0, overdue: 0 }
  for (const r of rows) byStatus[r.status]++
  const people = new Map<string, Report['load'][number]>()
```

```
  for (const r of rows) {
    const p = people.get(r.assignee) ?? { assignee: r.assignee, open: 0, overdue:
    0, points: 0 }
    if (!r.done) {
      p.open++
      p.points += r.complexity
      if (r.status === 'overdue') p.overdue++
    }
    people.set(r.assignee, p)
  }
  return {
    total: rows.length,
    byStatus,
    top3: rows.filter((r) => !r.done).slice(0, 3),
    overdue: rows.filter((r) => r.status === 'overdue'),
    load: [...people.values()].sort((a, b) => b.points - a.points ||
    a.assignee.localeCompare(b.assignee)),
    completion: rows.length ? Math.round((100 * byStatus.done) / rows.length) : 0,
  }
}

/** Демонстраційні задачі команди; дедлайни — відносно дня відкриття */
export function demoTasks(on: string): Task[] {
  const at = (days: number) => new Date((dayNumber(on) + days) *
    DAY).toISOString().slice(0, 10)
  return [
    { id: 't1', title: 'Налаштувати CI/CD для вебзастосунку', assignee: 'Андрій',
    deadline: at(2), importance: 5, complexity: 4, done: false },
    { id: 't2', title: 'Макет головної сторінки', assignee: 'Олена', deadline:
    at(-2), importance: 4, complexity: 3, done: false },
    { id: 't3', title: 'Модель даних і міграції БД', assignee: 'Тарас', deadline:
    at(6), importance: 5, complexity: 5, done: false },
    { id: 't4', title: 'Сторінка «Про нас»', assignee: 'Олена', deadline: at(12),
    importance: 2, complexity: 1, done: false },
    { id: 't5', title: 'Автентифікація користувачів', assignee: 'Андрій',
    deadline: at(-5), importance: 5, complexity: 4, done: true },
    { id: 't6', title: 'Тест-кейси для форми заявки', assignee: 'Марія', deadline:
    at(1), importance: 3, complexity: 2, done: false },
    { id: 't7', title: 'Оновити README', assignee: 'Тарас', deadline: at(-1),
    importance: 2, complexity: 1, done: false },
    { id: 't8', title: 'Демо для замовника', assignee: 'Марія', deadline: at(20),
    importance: 4, complexity: 2, done: false },
  ]
}
```